

tass — bot v1.4.1-nostarve

The agent that broke the Gold rank-200 ceiling (arena rank 392 → 187 / 530).
CodinGame Spring Challenge 2026 · "Troll Farm" · 2026-07-04 · a two-troll production ("gold-elite") strategy.

1. One-paragraph summary

v1.4.1 is a greedy, two-troll wood-production bot. It trains exactly one dedicated chopper and keeps both trolls productive every single turn. The starter funds and then continuously re-seeds a tight cluster of fast-growing BANANA trees next to the shack; the chopper fells those trees in a perpetual local loop, banking wood (worth **4x** a fruit) whenever it fills. The decisive feature that this version adds over its predecessor — and the reason it broke rank 200 — is the **anti-starvation rule**: a troll *never idles*. When its local farm is momentarily empty, it falls back to felling the nearest reachable tree anywhere on the map rather than parking. Eliminating zero-value turns lifted the head-to-head win rate against the production-heavy Gold field by 7–11 percentage points, which the ranking system converted into a ~160-place climb.

2. Why this strategy — the game's economics

The referee scores a player as $\sum \text{fruit_in_shack} + 4 \cdot \text{wood_in_shack}$ (iron scores 0). Wood is 4x leveraged, so a bot that converts its effort into wood out-scores a fruit farmer. Trees grow a "size" each cooldown tick, then bear fruit; a felled tree yields $\min(\text{size}, \text{free_carry})$ wood and is gone. The relevant per-type facts the algorithm exploits:

Species	Grow cooldown	Health = base + slope·size	Why the bot cares
BANANA	6 (→4 by water)	2 + 1·size (very low)	Grows fastest AND is cheapest to fell (≈2 chops at size 2) → the ideal crop. Worthless for training, so pure surplus.
APPLE	9 (→2 by water)	8 + 3·size (tanky)	Water-apple grows very fast but is slow to fell — used as a secondary crop only near water.
PLUM/LEMON	8 (→3)	4 + 2·size	Training currency (troll cost = $n + \text{stat}^2$); harvested only to fund the chopper.

The winning insight (decoded from the top Gold bots GoodDevel & PonyPonyCodeCode): 2 trolls, harvest-first opening, then a **banana printer + perma-chopper** banking ~40–70 wood by turn 300 (= 160–280 points). Our earlier denial/hybrid bots banked ~26 wood and lost the production race; matching this meta is what reached Gold, and refining its efficiency is what climbs it.

3. Architecture: two trolls, fixed roles

The bot commits to **exactly two trolls** ($\text{GE_MAX} = 2$): the (1,1,1,1) **starter** it begins with, plus **one trained chopper** with talents (ms 2, cc 2, hp 0, chop 2) — abbreviated (2,2,0,2). Role is decided purely by $\text{chop_power} \geq 2$.

Troll	Talents	Job
Starter	(1,1,1,1)	① harvest fruit + mine iron to <i>fund</i> the chopper (turns 1–~20); ② after the chopper exists, run the banana printer : pick/harvest bananas and re-plant them near base; ③ help chop (it has chop 1) when the farm is stocked.
Chopper	(2,2,0,2)	Perma-fell local trees; bank on fill. cc 2 means it fills after 2 wood → short trips. hp 0 (can't harvest) makes it cheaper (saves apple), and it never needs to.

Why not the boss's expensive cc-4 chopper or 3-4 trolls? Both were measured worse: a cc-4 chopper drains the economy (lemon cost $n+16$), and every alternative troll count / spec loses to the 2-troll `(2,2,0,2)` configuration in head-to-head simulation.

4. Training policy

```
// once per game: train the ONE chopper as soon as it is affordable, then stop.
want_chopper = (troll_count < 2) && (no troll has chop ≥ 2)
cost          = training_cost(n, (2,2,0,2)) // n + ms², n + cc², n + hp², n + chop² (iron)
train_now     = want_chopper && can_afford(cost)
need_iron     = want_chopper && iron < cost.iron && fruit_already_covers_cost
need_fund[t]  = inv[t] < cost[t] // which of plum/lemon/apple still block the chopper
```

Until the chopper is affordable the starter is steered by `need_fund` (harvest the missing fruit types) and `need_iron` (mine iron). This funds the chopper around turn 13-25 on normal maps — matching the decoded elite's timing.

5. Per-turn decision flow

Each turn, for every troll (sorted by id) the bot runs a fixed priority cascade and emits one command. A per-troll BFS distance map `d` over walkable cells is computed once; a `reserved` set prevents two trolls from targeting the same cell.

5a. Chopper cascade

```
CHOPPER (chop ≥ 2):
1. endgame? (turns_left ≤ dist_home+2 with a carried load) → bank it
2. full (free == 0) → bank
3. standing on a felleable tree (size ≥ fell_size) → CHOP
4. a felleable own-half tree in roam range exists → MOVE toward the nearest/fastest
5. ★ ANTI-STARVATION: farm empty → nearest reachable tree → MOVE/CHOP it (size ≥ 1, anywhere)
6. carrying something → bank the partial load
7. otherwise → park by the shack (last resort)
```

5b. Starter cascade

```
STARTER (chop 1):
1. endgame banking (same rule)
2. carrying a banana & farm has room → walk to a free base cell & PLANT it (printer, step B)
3. full → bank
4. on a ripe tree we want → HARVEST (funding: deficit fruit; post-fund: only banana/water-apple seeds)
5. FUNDING PHASE: MINE iron / go harvest the fruit types the chopper still needs
6. BANANA PRINTER (post-funding):
   a. banked bananas at shack → PICK one (fastest seed cycle) (printer, step A)
   b. no banked seed → go HARVEST a native banana/water-apple to get one
7. farm full / no seeds → help CHOP (chop 1), else
   ★ (v1.4.2 also applies anti-starvation here), else park
```

6. The banana printer — the wood engine

The printer is a closed loop that keeps a dense cluster of ~fast-growing banana trees within `farm_r = 3` of the shack (cap `farm_max = 12`), so the chopper always has a ripe, low-health tree a step away — no travel, no idle.

1. **Seed acquisition (A):** the starter `PICK`s a banana out of the shack inventory (bananas banked from felling/harvesting), or if none are banked, `HARVEST`s a native banana tree to obtain one.

2. **Planting (B):** carrying a banana, it walks to the nearest free base cell — **preferring water-adjacent cells** (banana cooldown 6→4 beside water) — and `PLANT` s it.
3. **Felling:** the chopper fells these local bananas at `fell_size = 2` (2 wood, ≈2 chops), banks at cc 2, and the loop repeats. Banked fruit stays near zero — everything funnels into wood.

Species choice is deliberate: banana's cooldown-6 growth and health 2+size give the best wood-per-chop-time of any tree; water-apple is used only as a secondary seed when standing on one.

7. ★ The anti-starvation rule (why v1.4.1 broke rank 200)

The change. Previously, when a chopper had no felleable tree in its local farm (`nearest_fell` returned nothing), it fell through to `park` — it stood next to the shack doing nothing. On maps where the printer could not keep the farm stocked (banana-poor or contested starts) this produced the arena's shutdown games: **~5 plants, the chopper wandering, ~22 wood, a 115-point loss.**

```
// v1.4.1 fallback, inserted just before "park":
if local farm empty {
    tc = nearest reachable tree with size ≥ 1, minimising steps(travel) + ceil(health/chop)
    if tc exists { if on it → CHOP else → MOVE toward it } // 1 wood beats 0
}
```

Felling a size-1 tree for a single wood is worth far more than an idle turn. This converts the shutdown *losses* into competitive games. Because Gold rank is **win-rate driven** (not score margin), turning losses into wins is pure ranking gain.

Measured effect — head-to-head win rate against fixed sparring opponents, anti-starvation OFF → ON:

Opponent (archetype)	OFF	ON	Δ
schedbot (production)	52.7%	63.3%	+10.6
scriptboss (wood-race boss)	70.0%	78.7%	+8.7
silverboss (mixed)	91.0%	98.0%	+7.0
mybot (denial)	74.7%	77.7%	+3.0

Arena result: Gold rank 392 → 187 (crossed the rank-200 line); rating 11.59 → 15.9. (A self-play A/B masks this: the env flag flips *both* mirror sides, so the gain only shows against a fixed third party — a measurement subtlety worth remembering.)

8. Safety mechanisms

Mechanism	What it prevents
Endgame banking — any carried load is banked when <code>turns_left ≤ eta_home + 1</code>	Carried goods score 0 at t=300; a chopper's stranded wood is 4 pts each.
Anti-stall watchdog — if a troll issued a MOVE but its position is unchanged for 2+ turns, it sidesteps to a free adjacent cell	The #1 arena loss cause in earlier bots: a troll spamming an identical blocked MOVE for dozens of turns behind its own teammate/crop.
Reserved-cell set	Two of our trolls targeting/standing on the same cell (referee forbids it; would waste a move).
Shack-adjacent DROP cell (never the shack itself)	The shack tile is unwalkable; a troll must DROP from an orthogonally-adjacent cell.

9. Tunable constants (arena defaults)

Constant	Default	Meaning
GE_SPEC	(2,2,0,2)	the one chopper's talents (ms,cc,hp,chop)
GE_MAX	2	total troll cap
GE_FARM_R	3	banana-farm radius around the shack
GE_FARM_MAX	12	max base trees the printer maintains
GE_FELL_SIZE	2	min tree size the chopper fells normally (1 in liquidation / anti-starvation)
GE_CHOP_R	99	max shack-distance the chopper roams for a normal fell
GE_LIQ_T	34	final turns: fell anything reachable (liquidate standing wood)
GE_STARTER_CHOP	1	let the starter help chop when the farm is stocked

All of these were swept; the defaults are a verified local optimum — no single knob beats a mirror of the default configuration.

10. Where the code lives

File	What
cgauto/submissions/v1.4.1-nostarve.rs	the exact submitted single-file bot (minified, stdlib-only, compiles with <code>rustc --edition 2021 -O</code>)
docs/v1.4.1-nostarve-readable.rs	readable un-minified reconstruction of the same bot (verified: compiles & reproduces the sample output)
rust/src/strategies/gold_elite.rs	the core algorithm as a library strategy, heavily commented — the clearest place to read it (current tip = v1.4.2, which additionally applies the anti-starvation rule to the starter)

The live decision function inside the single file is `fn decide_elite(state: &State) -> Vec<String>`. Generated 2026-07-04. The exhaustive experiment history is in `docs/silver-experiment-log.md`.